

HCRTOS SDK pwm userguide

1. Table of

HCRTOS SDK pwm userguide

- 1. Table of Contents
- 2. Documentation History
- 3. Overview
 - 3.1 Purpose of Writing
 - 3.2 Audience
- 4. Module Introduction
- 5. Module Interface Description
 - 5.1 Interface Functions
- 6. Module Test Cases and Sample Code
- 7. Module Debug Methods
- 8. Frequently Asked Questions

2. Documentation

Version Number	Date	Created/Revised Person	Create/Revision History
1.0	2023.04.10	Qiu Haojia	Added Explanatory Document
2.0	2023.04.23	Qiu Haojia	Supports setting duty cycle and period during initialization
3.0	2023.04.25	Qiu Haojia	Cancel setting duty cycle and period during initialization, and avoid overlap at different stages Sub-initialization
4.0	2023.08.14	Qiu Haojia	Added Polarity Setting
5.0	2023.09.09	Qiu Haojia	Added Configurable PWM Range Selection

3. Overview

3.1 Purpose of writing

Guidance and introduction to the use and development of PWM

3.2 Target readers

Software development engineers and technical support engineers.

4. Module

- The RTOS PWM module distinguishes between the 15xx series and 16xx series chips; the 15xx series has 3 PWM channels, the 16xx series has 6 PWM channels.
- In the current driver, the maximum PWM clock is 27M, and the maximum period at 27M clock is 37ns.
- Supports setting duty cycle and period during driver initialization.

4.1 Device Tree Configuration

Taking pwm0 as an example:

```
1 | pwm@0 {  
2 |     pinmux-active = <PINPAD_L23 1>;  
3 |     devpath = "/dev/pwm0";  
4 |     polarity = <1>;  
5 |     status = "okay";  
6 | };
```

pinmux-active: Configure the pin PINPAD_L23 to PWM mode. It does not mean that all PWM values are set to 1. Please refer to the specific configuration for the pins used.

references: components/kernel/source/include/uapi/hcuapi/pinmux/hc15xx_pinmux.h
and components/kernel/source/include/uapi/hcuapi/pinmux/hc16xx_pinmux.h.

devpath: Path of the generated PWM node.

polarity: Polarity of the channel after initialization. 0: low level; 1: high level.

status: 'okay' means enabled; 'disabled' means not enabled.

PWM duty cycle and frequency are set through code; please refer to the sample code.

4.2 Menuconfig configuration

In the SDK root directory, run make menuconfig and select PWM under the following path.

```
1 | Location:  
2 |     -> Components  
3 |         -> kernel (BR2_PACKAGE_KERNEL [=y])  
4 |             -> Drivers
```

```
-----). Highlighted letters are hotkeys. Pressing <Y> selects a feature, while <N> excludes a feature. Press <E>

Drivers

[*] Uart driver --->
[ ] Virtual uart driver ----
[ ] File uart driver ----
[*] Dummy uart driver
[*] Enable /dev/null
[ ] Enable /dev/zero
-- Softerq support
-- Lnx support --->
[*] Workqueue support --->
[ ] AMP RPC support ----
[*] Ramdisk
[*] RAM disk wrapper (mkrd)
[*] Block-to-Character (BCH) Support --->
[*] Memory Technology Device (MTD) Support --->
[ ] FIFO and named pipe drivers ----
  Buffering --->
[*] mmz dev
[*] video --->
[*] pinmux
[*] gpio
[*] ADC Support
[*] Audio Support --->
[*] Video Support --->
[*] avsync dev Support
[ ] poH Support
[*] input event --->
[*] spi support --->
[*] lvdls --->
[*] mapi
-- I2C Driver Support --->
[*] PWM Driver Support ---
[ ] NB debug
[*] audio platform support ----
[ ] watchdog ----
[*] persistent memory
-- hwspinlock
(+)
```

4.2.1 PWM frequency range selection

To meet PWM having 0-100% and each adjustable level at 1%, you can set values within the PWM range, but you can also set frequencies beyond the maximum of the range; however, the adjustable level becomes uncertain and varies with the configured frequency.

When only one channel is used, CONFIG_RANGE_AUTO_SET can be configured, and the division factor will be calculated automatically. However, beyond the range of this division factor, the adjustable level is uncertain.

> Components > >kernel > Drivers > PWM Driver Support

About How to calculated PWM frequency

$$\text{freq} = 2 / (\text{div} * (\text{h_val} + \text{l_val}));$$

$$\text{hval} = \text{duty_ns} * \text{div} / 373$$

$$\text{lval} = (\text{period_ns} - \text{duty_ns}) / \text{div} / 373$$

T6 meet the adjustable level of 2%

the h_val and l_val have following requirenents:

$$\text{Max}(\text{h_val} + \text{l_val}) = 65535;$$

$$\text{Min}(\text{h_val} + \text{l_val}) = 266;$$

the div can Set in make menuconfig:

CONFIG_41Hz_270K : div = 1;

CONFIG_div : div = 2;

CONFIG_RANGE_103Hz_67KHz : div = 4;

CONFIG_RANGE_50Hz_33KHz : div = 8;

CONFIG_RANGE_AUTO_SET : when use only one channel ,
div uill auto calculated ta meet with the frequency you set;

if div = 1; the range is 411(Hz)-270(KHz),

you can set bigger than 276(KHz), but adjustable level may not be 1%5

Prompt; Select pwm frequency range

Location:

-> Components

-> kernel (882 _PACKAGE_KERMEL [=y])

> Drivers

-> PWM Driver Support (CONFIG_PWM_ORIVER [=y])

Defined at 16

Depends on: 8R2_ PACKAGE_KERMEL [-y] & CONFIG DRIVER [-y]

Selected by [n:

BR2_ PACKAGE_KERIEL [=y] & eye CONFIG ORIVER [-y] BLA n

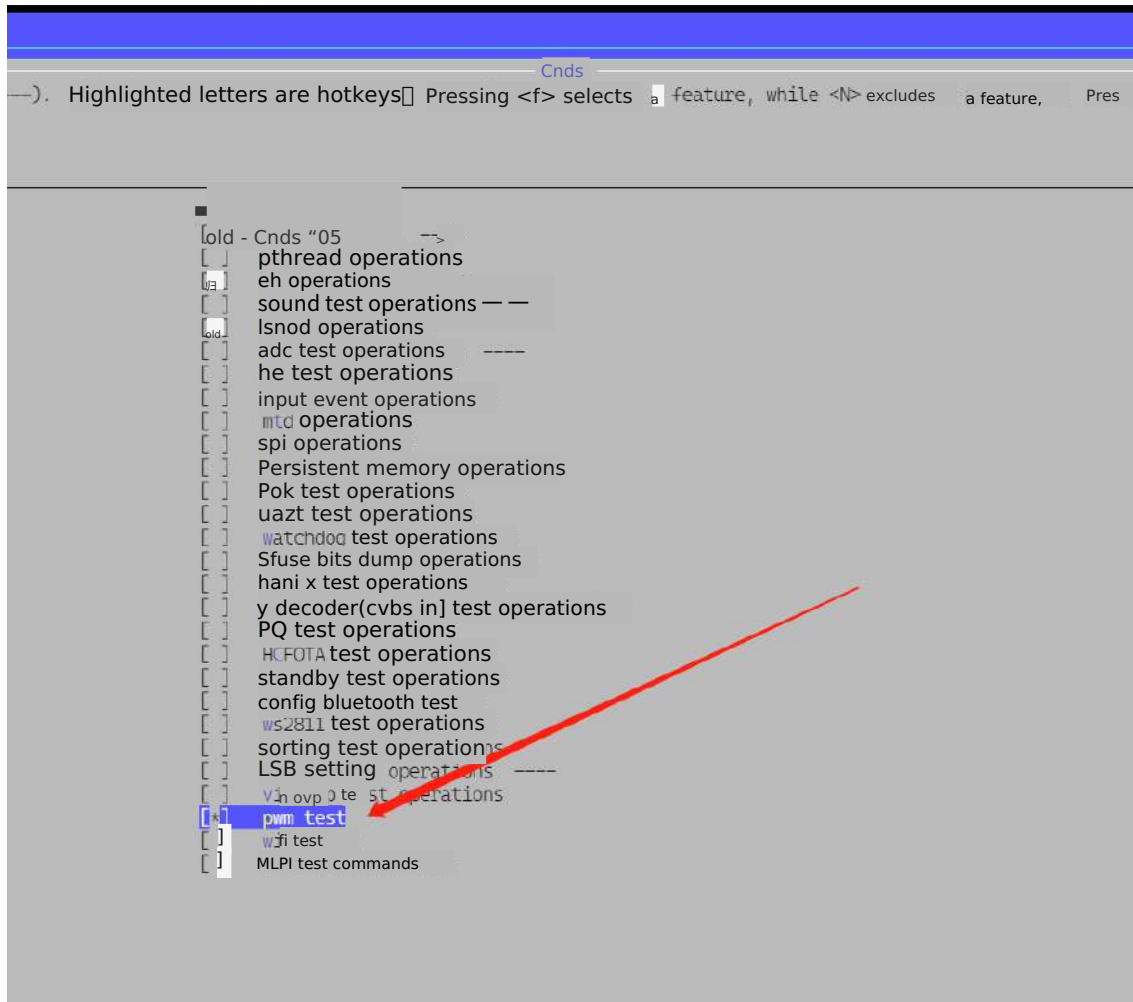
After configuration is complete, enter make kernel-rebuild all to compile and burn,
and enter the following commands in the serial console to view the pwm node.

```
hc1512aGdbB266#
hc1512aGdbB200# nsh
hc1512a@dbB266 (nsh)# ls
/:
dev/
hc1512a@dbB200 (nsh)# cd dev
hc1512aGdbB269 (nsh)# ls :
auddec
audsink
avsync0
avsync1
bus/
dis
fb0
ge
input/
mmz
mtdblock6
mtdblock1
mtdblock2
mtdblock3
mtdblock4
null
pers) s+ pntmem
Pwm6
st_prodect
sndC0i2so
uart1
uart_dummy
viddec
vidsink
hc1512a@dbB200 (nsh)# phase
```

4.3 test command

In the SDK root directory, enter make menuconfig and select the pwm_test command according to the path below.

- 1 Location:
- 2 -> Components
- 3 -> Cmds (BR2_PACKAGE_CMDS [=y])



After configuration is complete, enter make cmds-rebuild all to compile and burn, and input the following commands in the serial console to enable test commands.

```
hc1512a@dbB200#
hc1512a@dbB200#
hc1512a@dbB200# pwm_test -h
Usage: pwm_test
-s --start          start pwm device id
-p --period_ns      set period_ns
-d --duty_ns        set duty_ns
-P --ploarity       set ploarity
-S --stop           stop pwm device id

Invalid command "pwm_test"
hc1512a@dbB200# pwm_test -s0 -p1000000 -d400000 -P0
start /dev/pwm0 test
hc1512a@dbB200#
```

5. Module interface

5.1 Interface functions

This module provides ioctl function interfaces for the application layer.

- Set PWM parameters, including duty cycle, period, and polarity. Note that the units for duty cycle and period are ns.

```
1 info.polarity = polarity;
2 info.period_ns = period_ns;
3 info.duty_ns = duty_ns;
4 ioctl(fd, PWMIOC_SETCHARACTERISTICS, (unsigned long)&info);
```

- Get PWM parameters, including duty cycle, period, and polarity. Note that the units for duty cycle and period are ns.

```
1 info.polarity = polarity;
2 info.period_ns = period_ns;
3 info.duty_ns = duty_ns;
4 ioctl(fd, PWMIOC_GETCHARACTERISTICS, (unsigned long)&info);
```

- Start and stop PWM.

```
1 ioctl(fd, PWMIOC_START);
2 ioctl(fd, PWMIOC_STOP);
```

6. Module test cases and sample code

Code is stored at: components/cmds/source/pwm/pwm_test.c. Note that the units for duty cycle and period are ns.

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <fcntl.h>
4 #include <poll.h>
5 #include <signal.h>
6 #include <stdlib.h>
7 #include <string.h>
8 #include <getopt.h>
9 #include <sys/ioctl.h>
10 #include <hcuapi/pwm.h>
11 #include <kernel/lib/console.h>
12
13 static void print_usage(const char *prog)
14 {
15     printf("Usage: %s \n", prog);
16     puts("    -s --start          start pwm device id\n")
17     "    -p --period_ns    set period_ns\n"
18     "    -d --duty_ns      set duty_ns\n"
19     "    -P --polarity     set ploarity\n"
20     "    -S --stop         stop pwm device id\n");
21 }
22
23 static int pwm_test(int argc, char *argv[])
24 {
```

```

25     int fd;
26     int id = 0;
27     bool stop_t = false;
28     char path[64] = { 0 };
29     struct pwm_info_s info = { 0 };
30     uint32_t period_ns, duty_ns, polarity;
31     period_ns = 1000000;
32     duty_ns = 500000;
33     polarity = 0;
34
35     opterr = 0;
36     optind = 0;
37
38     while (1) {
39         static const struct option lopts[] = {
40             { "start",                1, 0, 's' },
41             { "period_ns",            1, 0, 'p' },
42             { "duty_ns",              1, 0, 'd' },
43             { "stop",                 1, 0, 'S' },
44             { "polarity",             1, 0, 'P' },
45             { "NULL",                 0, 0, 0 },
46         };
47
48         int c;
49
50         c = getopt_long(argc, argv, "s:p:d:S:P:", lopts, NULL);
51
52         if (c == -1) {
53             break;
54         }
55
56         switch(c) {
57             case 's':
58                 id = atoi(optarg);
59                 break;
60             case 'p':
61                 period_ns = atoi(optarg);
62                 break;
63             case 'P':
64                 polarity = atoi(optarg);
65                 break;
66             case 'd':
67                 duty_ns = atoi(optarg);
68                 break;
69             case 'S':
70                 stop_t = true;
71                 id = atoi(optarg);
72                 break;
73             default:
74                 print_usage(argv[0]);
75                 return -1;
76         }
77     }
78
79     sprintf(path, "/dev/pwm%d", id);
80     fd = open(path, O_RDWR);
81     if (fd < 0) {
82         printf("%s    open fail\n", path);

```

```

83         return 0;
84     }
85
86     if (stop_t == true) {
87         printf("stop      %s test\n", path);
88         ioctl(fd, PWMIOC_STOP);
89         close(fd);
90         return 0;
91     }
92
93     info.polarity = polarity;
94     info.period_ns = period_ns;
95     info.duty_ns = duty_ns;
96     ioctl(fd, PWMIOC_SETCHARACTERISTICS, (unsigned long)&info);
97
98     ioctl(fd, PWMIOC_START);
99     close(fd);
100    printf("start      %s test\n", path);
101
102    return 0;
103 }
104
105    console_cmd(pwm_test, NULL, pwm_test, console_cmd_mode_self, "pwm test
operations")

```

7. Module debugging

Can use an oscilloscope or logic analyzer to capture waveforms for viewing.

8. Frequently Asked

None for now.